

WebTrail 项目讲解稿（按 PPT 逐页）

共 18 页，建议时长 10-15 分钟。

第 1 页 · 封面

大家好。

今天向大家介绍一个我开发的开源工具，叫 **WebTrail**——一款浏览器数字取证与用户画像工具。

它的定位很清晰：用纯 Python 标准库实现，零第三方依赖，拷一个文件夹就能跑。提供命令行和图形界面两种操作方式，跨 Windows、macOS、Linux。

第 2 页 · 项目概述

先说一下 WebTrail 是什么。

它是一款面向数字取证的工具，能一键提取你电脑上 Chrome、Edge、Firefox 的全部浏览痕迹，然后自动把这些碎片化数据转化为结构化的调查情报。

目标用户是安全分析师、取证调查员和 IT 审计人员。它的核心价值就四个字：**提取加分析一体化**——数据采集到报告输出一条链路。纯标准库实现，不需要 pip install 任何东西。

第 3 页 · 背景与动机

为什么要做这个工具？三个痛点。

第一，工具碎片化。 市面上商业取证工具要么贵、要么部署复杂、要么各种捆绑安装。我需要一个便携的、绿色的方案。

第二，信息过载。 大部分同类工具只做到“提取”，调查者就拿着几千条原始记录一条条看，效率很低。

第三，缺乏画像能力。 我更希望这个工具能自动帮我回答三个问题：这个人什么时候最活跃、常逛什么网站、有没有异常行为。

所以 WebTrail 的答案是：一个命令把证据固定→数据提取→画像分析→报告输出全部做完。

第 4 页 · 核心功能总览

四大能力模块一目了然。

痕迹提取——浏览历史、Cookie、下载记录、书签、登录凭证（也就是密码）、缓存元信息，共六类。

浏览器支持——Chrome、Edge、Firefox。Edge 和 Chrome 都是 Chromium 内核，所以我们用继承复用了 Edge 的代码，后面会单独讲。

取证保障——只读访问、SHA-256 链式保管、记录级完整性校验、时间戳保真。

画像分析——24 小时活跃热力图、TOP 域名排名、8 类 URL 语义分类、风险指标自动检测。

第 5 页 · 技术架构总览

整个项目 15 个源文件，大约 1800 行代码，分五层。

入口层： `main.py`，87 行，解析 CLI 参数，决定走命令行还是图形界面。

管道层： `pipeline.py`，119 行，定义标准三阶段取证流程，是整个工具的编排中枢。

适配器层： `browsers` 目录，Chrome、Edge、Firefox 各一个文件。用 ABC 抽象基类来约束接口，Edge 直接继承 Chrome。

分析层： `core` 目录，`profiler.py` 做七维度画像分析，`timeline.py` 构建时间线。

输出层： `output/writer.py`，生成 JSON 取证报告和 CSV 表格。

这个架构遵循关注点分离原则：改浏览器适配器不影响分析逻辑，改 GUI 不影响命令行。

第 6 页 · 三阶段取证管道

这是整个工具最核心的设计——三道不可跳跃的阶段。

阶段 1：证据哈希。 提取数据前，先把所有 `.sqlite` 和 `.json` 证据文件做 SHA-256 哈希。如果将来有人说数据被篡改过，用这条哈希链就可以反证。

阶段 2：痕迹提取。 每个浏览器适配器并行运行，统一成 `ArtifactRecord` 数据结构，错误隔离——Chrome 挂了不影响 Firefox 继续跑。

阶段 3：用户画像。 基于全部痕迹做七维度分析，生成 JSON 加 CSV 报告。

三个阶段，环环相扣，任何一个阶段失败了都可以回溯。

第 7 页 · 阶段 1：证据哈希

展开看一下第一阶段。

代码很简单，遍历所有浏览器配置目录，找出 `.sqlite` 和 `.json` 文件，逐个计算 SHA-256。刚在这台机器上跑是 189 个文件。

右边是哈希链验证流程：从证据文件开始，哈希记录在 `report.json` 元数据里，报告本身再有自校验文件 `report_checksums.json`。整条链从文件到报告，贯穿始终，保证溯源能力。

第 8 页 · 阶段 2：痕迹提取

第二阶段，痕迹提取。

我们定义了一个核心数据结构叫 `ArtifactRecord`，用 Python 的 `dataclass` 实现，8 个字段——类型、浏览器、ISO 8601 时间戳、配置名、源文件路径、数据载荷、提取时间和本记录的校验和。

六大提取类型——History 对应 SQLite 的 `urls` 表、Cookie 对应 `cookies` 表、Download 对应 `downloads` 表、Bookmark 是 JSON 递归解析、Login 对应 `Login Data` 表、Cache 是目录元信息加 HSTS。

安全保障：所有 SQLite 连接都是只读模式，浏览器运行时数据库被锁会自动复制到临时文件再读，读取完毕后自动清理。每条记录失败不中断全局。

第 9 页 · 阶段 3：用户画像

第三阶段，用户画像，七维度分析结果。

`overview` 总览——记录总数、时间范围、浏览器分布。`activity_heat` 活跃热力图——24 小时分布加 30 天逐日统计。`top_domains` 域名排名。`top_categories` URL 八类语义分类。`behavior_insights` ——高峰时段、夜间占比、登录数量。`browser_usage` ——各浏览器按类型细分。`risk_indicators` ——风险指标。

这七个维度覆盖了调查者在做用户画像时最常见的需求。

第 10 页 · 浏览器适配器设计

来看适配器层的设计。

基类是 `BaseExtractor`，ABC 抽象类，定义了 8 个抽象方法——`detect_profiles`、`extract_history`、`extract_cookies` 等。`run` 方法是模板方法，统一调用这些抽象方法并做错误捕获。

三个实现类：Chrome 212 行，扫描 Default 和 Profile N 目录，时间戳是 1601 年 epoch 的微秒数。Edge 18 行，仅覆盖 `__init__` 把浏览器名设为 Edge。Firefox 198 行，解析 `profiles.ini` 文件，时间戳是 1970 年的 PRTIME 微秒数。

第 11 页 · Edge 继承设计

这个页面专门展开讲 Edge 的继承设计。

Edge 只有 18 行代码，继承 `ChromeExtractor`，只改了 `self.browser = "Edge"` 这一行。

为什么可以这样？因为 Edge 基于 Chromium 内核，数据库格式和 Chrome 完全一样——History 表、Cookies 表、Login Data 表结构相同，时间戳也是 WebKit 微秒，书签也是同一套 JSON 结构。

这是「组合优于继承」原则的反例——当两个类 95% 的行为相同时，继承反而最优。如果不用继承，你需要把 Chrome 整整 212 行代码全部复制一遍，之后修一个 bug 要改两个地方。

第 12 页 · 取证科学性

这一页讲取证科学性保障，四个维度。

只读防污染。 所有 SQLite 连接 `mode=ro` 加 `PRAGMA query_only=ON`，被锁时自动 fallback 到临时文件，读完自动清理。

链式保管。 提取前对全部证据文件做 SHA-256，记录在报告元数据中，报告自身再有一次哈希自校验。

记录级完整性。 每条 ArtifactRecord 都带 checksum 字段，序列化为 JSON 后计算 SHA-256，可逐条独立验证。

时间戳保真。 Chrome 的 1601 微秒、Firefox 的 1970 微秒，全部转为 ISO 8601 标准格式，保留原始时区信息，不做截断。画像分析时自动转为本地时区。

第 13 页 · 风险指标检测

风险检测分为三类。

第一，隐私模式推断。 如果提取出来的记录总数少于 10 条，说明用户可能长期使用浏览器的无痕模式——无痕模式下浏览历史不会被持久化。

第二，历史清除检测。 Cookie 还在，但浏览历史为零。这说明用户手动清除了历史记录，是一个典型的反取证行为。

第三，可疑域名关键词匹配。 八个关键词——torrent、crack、keygen、warez、pirate、darkweb、onion、hacktool。遍历所有 URL，命中后记录关键词、完整 URL 和来源浏览器供人工研判。

底部加了免责声明：以上指标仅作参考，需结合上下文人工研判，这不是 AI 定罪。

第 14 页 · 工具层详解

`utils` 目录有四个支撑模块。

sqlite_utils.py: `safe_connect` 做只读连接，`safe_connect_with_fallback` 做锁定时自动回退，自定义 `_dict_factory` 让所有 Python 版本都能用 `.get()` 方法取字段。

time_utils.py: 三个时间转换函数——Chrome 微秒转 ISO、Firefox PRTime 转 ISO、Unix 毫秒转 ISO，统一输出。

url_utils.py: 域名提取支持去掉协议、端口、前导点（Cookie 中有 `.google.com` 这种格式）。URL 分类用预定义的八类规则字典，可扩展。

config.py: 跨平台路径适配，Windows、macOS、Linux 各自对应不同的 Chrome、Edge、Firefox 数据目录。

第 15 页 · 图形界面

GUI 用的是 Python 标准库自带的 tkinter，零第三方依赖。

左侧是控制面板：开始分析按钮、报告保存位置输入框加浏览按钮、进度条。分析点下去之后，`threading.Thread` 启动异步执行，UI 不冻结。

右侧五个选项卡——概览、TOP 域名、活跃时段、访问类别、风险指标。顶部标题栏自动检测三个浏览器的安装状态，可多选。

分析完成后，自动调用 `write_report` 把报告写到用户指定的目录，状态栏会显示完整保存路径。一个按钮搞定全部流程。

第 16 页 · 报告输出

报告目录下生成四个文件。

`forensic_report.json` ——元数据加七个维度画像加全部痕迹记录，是完整的取证数据包。

`records.csv` ——UTF-8 BOM 编码的扁平化表格，Excel 双击就能打开分析。

`timeline.csv` ——按时间升序排列的活动事件，方便做时序分析。

`report_checksums.json` ——前三个文件各自的 SHA-256 哈希，用于防止报告文件在传输或存储中被篡改。

第 17 页 · 实际效果

这是在这台电脑上实际跑的结果。

总共 1573 条痕迹记录，时间线事件 1570 条，从 2022 年 12 月 20 号一直到今天。Chrome 1379 条，Edge 191 条，Firefox 3 条，全部零错误。证据文件 189 个，全部完成 SHA-256 哈希。

TOP 域名包括 `fanyi.baidu.com`、`github.com`、`chatgpt.com`、`claude.ai`，类别主要是搜索引擎和技术开发。活跃高峰在下午，这是一个典型的技术人员画像——平时大量使用翻译和代码工具。

第 18 页 · 总结

最后总结一下。

WebTrail 的核心价值是「提取加分析」一体化——不是给你扔一堆 JSON 就完了，而是一整条从证据固定到报告输出的完整链条。

15 个源文件，约 1800 行代码，纯标准库，零依赖。设计上遵循关注点分离、继承复用和最少依赖原则。

如果你需要支持新浏览器，继承 `BaseExtractor` 就行。要加新分析维度，在 `profiler` 里添一个函数。要新输出格式，在 `writer` 里加一个方法。架构已经把接口都预留好了。

以上就是 WebTrail 的全部介绍。感谢聆听，欢迎提问。